

bench4JSBACH Documentation

Release 0.0.1

Deike Kleberg

Oct 14, 2016

Contents

1 Introduction 1

2 User guide for benchmarking 3

3 User guide for developing diagnostic fields 11

4 Code documentation 17

Bibliography 26

Python Module Index 27

Index 28

Contents:

Introduction

Main idea

With **bench4jsbach** we started developing a tool for the scientists of the land department in the Max Planck Institute for Meteorology to make the comparison of JSBACH model simulation output data *[ReJSBA]* with one another and with data products straight forward and user friendly. In the first instance the tool shall serve for the benchmarking of land models, it has to be able to quantify the deviation of a simulation from reference experiments in the most crucial

and/or influential parameters out-of-the-box. This will help developing models as well as managing a larger number of them. It is desirable to cover most research areas of the land department and to keep the tool well adapted to the local technical conditions at the Max Planck Institute for Meteorology.

In order to have a living, dynamical tool, the possibility for individual extension is provided to the user from the beginning. This will help the benchmarking process to retain its validity but also help the scientists to handle future questions bringing new values into focus. It may also foster the communication between the groups as individual developments can easily be made usable for the whole department and thus serve as a basis for a fruitful discussion on the most significant diagnostic for the generally applicable benchmarking in the department.

Concepts

A guiding idea for **bench4jsbach** is the separation of issues that strongly depend on the origin of the climate data from issues directly related to their comparison. It is intended to solve the origin dependend tasks 'under the hood' of the tool so that a scientist can focus on the diagnostic and benchmarking methods.

Experiments

We use the generic term `experiment` independent from the data origin holding earth system model simulation output or observational data. Origin dependend processing like locating and extracting necessary raw data from different host, filesystem and out of datafile but also data interpolation are gathered under the name.

You may assign the role of a reference or that of a candidate to an `experiment`, in a comparison at least one instance of each is needed. It is possible to have multiple `experiment` s and also a single instance can take both roles in one **bench4jsbach** run. The main difference of the roles is the comparison being made and the report being written from the perspective of the candidate `experiment` s. Depending on the evaluation method each candidate can be compared with single references or with all references at once.

Diagnostic fields

The term `diagnostic field` clusters processing for the comparison of `experiment` s with a defined set of variables featuring a particular scientific question. It should contain at least one diagnostic and one benchmarking method. In this context the term benchmarking refers to a method that condenses the deviation of two experiments to one number, i.e. yields a global and time average value for it.

Context

The following diagram shows **bench4jsbach** in context with other actors (users, software, ...).

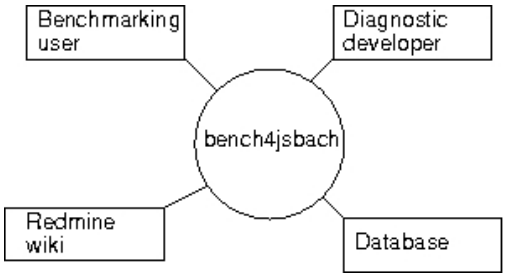


Fig. 1: **Figure:** Context diagram of **bench4jsbach**

Benchmarking user

- scientist managing many models
- model developer making still-in-line-check

Diagnostic developer dedicated scientist integrating a `diagnostic field`

Database database for reference `experiment` s, for the time being a table on the redmine project wikipage

Redmine wiki communication platform for **bench4jsbach**

Status

This documentation is based on a not yet tagged version (*0.0.1*) of **bench4jsbach**, a vertical prototype. The basic operating concept, the program structure and the workflow is implemented. The tool handles `experiment` s of type MPIESM1 (Max Planck Institute for Meteorology Earth System Model, *[GiESM1]*) and a `diagnostic field` analysing albedo.

Experiment types and diagnostic fields available

diagnostic field, experiment type	MPIESM1
albedo	•
hydrology	

Diagnostics and benchmarks in field albedo

diagnostic	name	type
Seasonal mean difference	delta	map
Global and time deviation	delta_gtd	metric
Global and time mean	delta_gtm	metric

Diagnostics and benchmarks in field hydrology

User guide for benchmarking

This chapter shall enable you to use **bench4jsbach** for the comparison of `experiment` s with predefined `diagnostic field` s.

Prerequisites

bench4jsbach is developed for UNIX and LINUX operating systems. It is written in object-oriented Python and works with version 2.7. The packages NumPy and NetCDF4 and the 2D plotting library matplotlib are used which are contained in most scientific Python distributions like Anaconda, Miniconda or Canopy. Furthermore **bench4jsbach** makes use of the benchmarking tool pyCMBS and the climate data command line operators CDO, both inhouse-developments of the Max-Planck-Institute for Meteorology.

We use Anaconda and can highly recommand it, as it simplifies package management and deployment, see http://conda.pydata.org/docs/_downloads/conda-cheatsheet.pdf. From here on it is refered to in the installation process.

Make sure that your system uses Python from `opt/anaconda/bin` with `opt` being your Anaconda base directory. If necessary, download and install Anaconda, you can get it from http://docs.continuum.io/anaconda/install#linux-install, and put `opt/anaconda/bin` to your system PATH. Then update and install the necessary Python packages, ..

```
>>> conda update conda
>>> conda update anaconda
>>> conda install netcdf4
>>> conda install -c https://conda.binstar.org/scitools cartopy
>>> conda install geos basemap
>>> conda install gdal
```

.. CDO ..

```
>>> pip install cdo
```

.. and pyCMBS

```
>>> cd opt
>>> git clone https://github.com/pygeo/pycmbs
>>> cd pycmbs
>>> env OMP_NUM_THREADS=1 python setup.py install
```

Get bench4jsbach

Change to the root directory of your local JSBACH repository git@git.mpimet.mpg.de:jsbach.git, that is the one containing the `.git` directory. If you do not have the repository get it with

```
>>> git clone git@git.mpimet.mpg.de:jsbach.git
```

Therefore you need passwordless ssh and the access permission for the git server (contact reiner.schnur@mpimet.mpg.de). Switch to the git branch `jsbach − dkl`

```
>>> git checkout jsbach-dkl
```

You will also need passwordless ssh for the servers from which you want to read experiment data.

Being in the git branch `jsbach − dkl` you find the files for the tool under `git/jsbach/tools/bench4jsbach` with `git/jsbach` as your git root directory. The structure of the **bench4jsbach** base directory `bench4jsbach` you find initially is shown below.

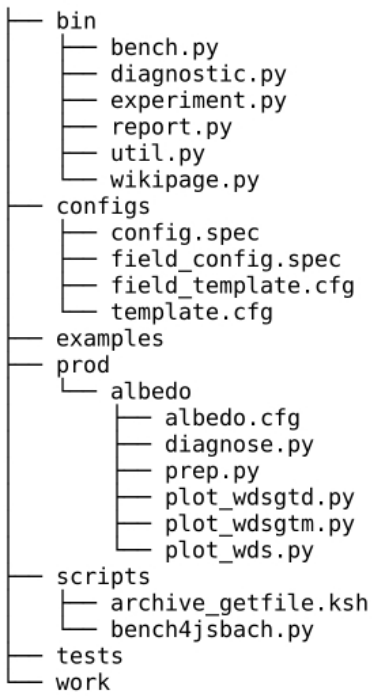


Fig. 2: **Figure:** initial structure of `git/jsbach/tools/bench4jsbach`

The following list describes the individual directories in `bench4jsbach`.

bin core source files

configs specification and template files for the run configuration

doc Sphinx documentation source files and Makefile

prod scripts and configurations for particular `diagnostic` `field` s

scripts essential parts of **bench4jsbach** e.g. run scripts

tests different testing environments

work currently hardcoded working directory

Prepare a run

A **bench4jsbach** run consists of two parts, the initial startup run which is done once to set up the environment and create the directory structure and the main run, fetching expriment data, analysing, making plots and writing a report. Both parts are controled by the run script `bench4jsbach/scripts/bench4jsbach.py`.

Initial run

A **bench4jsbach** run needs a name as it writes data to your local filesystem that has to be addressed. First choose this run name (e.g. `myb4jrun`) and change to the base directory `bench4jsbach`. Call the run script `bench4jsbach.py` with your run name as command line argument.

```
>>> cd git/jsbach/tools/bench4jsbach
>>> ./scripts/bench4jsbach.py myb4jrun
```

While running, the tool writes a welcome and a final success/error message to your display. In addition the internal debugger prints to standard output in the default setting.

```
****************************************
* BENCH4JSBACH:
* Creating environment and configfile for new bench4jsbach object 'myb4jrun'.
****************************************

### DKL_LOGGER - DEBUG -  2016/10/04 - 18:12:08 - MODULE bench - FUNCTION __init__ - LINE 143 :
  # Starting.
###

### DKL_LOGGER - DEBUG -  2016/10/04 - 18:12:08 - MODULE bench - FUNCTION _writeconfig__init__ - LINE
  # Starting.
###

### DKL_LOGGER - DEBUG -  2016/10/04 - 18:12:08 - MODULE bench - FUNCTION _writeconfig__init__ - LINE
  # Exiting.
###

### DKL_LOGGER - DEBUG -  2016/10/04 - 18:12:08 - MODULE bench - FUNCTION __init__ - LINE 147 :
  # Exiting.
###

****************************************
* BENCH4JSBACH:
* Created environment
*        '/home/git/jsbach/tools/bench4jsbach/work/myb4jrun'
* and the configuration file
*        '/home/git/jsbach/tools/bench4jsbach/work/myb4jrun/configs/myb4jrun.cfg'.
* Please edit this file.
****************************************
```

Listing: Standard output after the initial run with argument `myb4jrun`

When the startup run has finished successfully, the directory structure for `myb4jrun` is created (as shown in the figure below) and an editable, default configuration file `myb4jrun.cfg` is written in `myb4jrun/configs`, also shown further down.

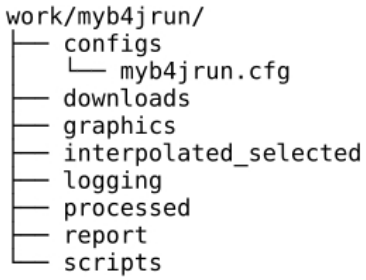


Fig. 3: **Figure:** Initial structure of `bench4jsbach/work/myb4jrun`

Main configuration file

The generated file is a form for the main configuration of your **bench4jsbach** run and has to be adapted to the planned experiment comparison. It is formatted to be read by a Python config file parser after your accomplishment. A config file consists of key value pairs and sections nested to any depth. For the sections square brackets are used, indentation clearifies the nesting level. You can write comments inline prepended by the hash symbol "#".

```
key-1=value-1
key-2=value-2
[section-1]
   key-1=value-1
   [[subsection-1]]
      key-1=value-1
```

Listing: Basic configuration file supported by a Python parser

After generation the file `myb4jrun.cfg` contains the structure of information that the configuration for a minimum **bench4jsbach** run with two `experiment` s (see *Experiments*) and one `diagnostic field` (see *Diagnostic fields*) has to provide.

In the file `myb4jrun.cfg`, which is displayed below, the mandatory fields to be filled in are marked with angle brackets (<,>) and located in the second part of the file, below the comment line "# PLEASE EDIT ...". The capital text within is meant to be explanatory and the whole string including the brackets has to be exchanged against your value for the respective item.

```
##################################################################
#
# configuration file for bench4jsbach object 'myb4jrun'
#
# automatically generated on 2016-09-22
#
##################################################################

### stamp like meta data, do not edit

bench_name=myb4jrun
author=deike
git_repository=origin(fetch):git@git.mpimet.mpg.de:jsbach.git,origin(push):git@git.mpimet.mpg.de:jsba
git_branch=jsbach-dkl
git_sha=167c9e4d3h5df92c0e8e8234878698b5956b35d1
git_status=modified
redmine_url=https://code.zmaw.de
redmine_project=jsbach-bench
redmine_wikipage=Bench4jsbach_simulations
host_name=scalar
host_ipaddress=127.0.1.1
b4j_base=/home/deike/git/jsbach/tools/bench4jsbach
timestamp=2016-09-22 11:26:48.446387

##################################################################
#
# PLEASE EDIT FROM HERE ON DOWNWARD
#
##################################################################

### global meta data

redmine_user=<REDMINE_USER>
redmine_key=<REDMINE_KEY>
```

```
institute="Max-Planck-Institute for Meteorology"
department="Land Department"
reporting=<BENCH_IS_REPORTING>

### experiments in diagnostic run, candidates and references

[experiments]
    [[<MODEL_1_ID>]]
        name_orig=<MODEL_1_NAME_ORIGINAL>
        type=<MODEL_1_TYPE>
        path=<MODEL_1_PATH>
        host=<MODEL_1_HOST>
        iscan=<MODEL_1_IS_CANDIDATE>
        isref=<MODEL_1_IS_REFERENCE>
        startyear=<MODEL_1_STARTYEAR_FOR_THIS_RUN>
        endyear=<MODEL_1_ENDYEAR_FOR_THIS_RUN>
        intitalyear=<MODEL_1_INITIALYEAR_OVERALL>
        finalyear=<MODEL_1_FINALYEAR_OVERALL>
    [[<MODEL_2_ID>]]
        name_orig=<MODEL_2_NAME>
        type=<MODEL_2_TYPE>
        path=<MODEL_2_PATH>
        host=<MODEL_2_HOST>
        iscan=<MODEL_2_IS_CANDIDATE>
        isref=<MODEL_2_IS_REFERENCE>
        startyear=<MODEL_2_STARTYEAR_FOR_THIS_RUN>
        endyear=<MODEL_2_ENDYEAR_FOR_THIS_RUN>
        intitalyear=<MODEL_2_INITIALYEAR_OVERALL>
        finalyear=<MODEL_2_FINALYEAR_OVERALL>

### diagnostic fields

[fields]
    [[<FIELD_1_NAME>]]
        config_filename=<FIELD_1_CONFIGURATION_FILENAME>
```

Listing: File `myb4jrun.cfg` after generation, a form for the main configuration

In the following the values to fill in are summarized.

Top level configuration

Top level in the main configuration file:

key		value	
name	description	datatype	default
redmine_user	redmine user login name	string	None
redmine_key	redmine “Api access key”	string	None
institute	users’ institute	string	None
department	users’ department	string	None
reporting	reporting switch	boolean	True

The redmine access is necessary if reference `experiment` metadata shall be retrieved from the table on the wikipage. When you are logged in you can get your redmine “Api access key” from the right menu on top of the page:

```
> My account > API access key > Show
```

If it does not exist yet, it can easily be created within the same menu.

Experiment section

The beginning of the section is marked by the header [experiments]. For each experiment for comparison in the run there has to be a subsection within, that is labeled with a run internal unique name.

Single experiment subsection:

key		value	
name	description	datatype	default
name_orig	original identifying infix, e.g. "vga0187"	string	-
type	experiment type	option (CMPI5, MPIESM1)	MPIESM1
path	None if item in database		
	path of experiment in filesystem on host	string	None
host	None if item in database		
	host of experiment	string	None
iscan	candidate role	boolean	-
isref	reference role	boolean	-
startyear	startyear for experiment in **bench4jsbach** run	string	-
endyear	endyear for experiment in **bench4jsbach** run	string	-
initialyear	initial year of experiment	string	None
finalyear	final year of experiment	string	None

Diagnostic field section

The beginning of the section is marked by the header [fields]. For each diagnostic field in the run has to be a subsection inside, that is labeled with a run internal unique name.

Single diagnostic field subsection:

key		value	
name	description	datatype	default
config_filename	diagnostic field configuration filename in bench4jsbach/prod/configs	string	-
	path to diagnostic field configuration file in local filesystem		

Files in context with the available diagnostic field s are stored in bench4jsbach/prod, the configuration files beneath in the directory configs. A list of available diagnostic field s can be found under *Experiment types and diagnostic fields available*.

When you have replaced all placeholders "<XXX>" with the appropriate values the bottom part of your configuration file should look in principle like the example below.

```
### global meta data

redmine_user='deike.kleberg'
redmine_key='XXX'
institute="Max-Planck-Institute for Meteorology"
department="Land Department"
reporting=True

### experiments in diagnostic run, candidates and references

[experiments]
```

```
    [[tra0163-0]]
        name_orig=tra0163
        type=MPIESM1
        iscan=False
        isref=True
        startyear=3300
        endyear=3329
    [[tra0163-1]]
        name_orig=tra0163
        type=MPIESM1
        iscan=True
        isref=False
        startyear=3970
        endyear=3999

### diagnostic fields

[fields]
    [[albedo]]
        config_filename=b4j_albedo.cfg
```

Listing: Extract of example configuration file `myb4jrun.cfg`

Run example myb4jrun

The run can now be started.

```
>>> ./scripts/bench4jsbach.py work/myb4jrun/myb4jrun.cfg
```

The CPU time maybe about one minute but, as it depends on the duration of the data fetching process, also more.

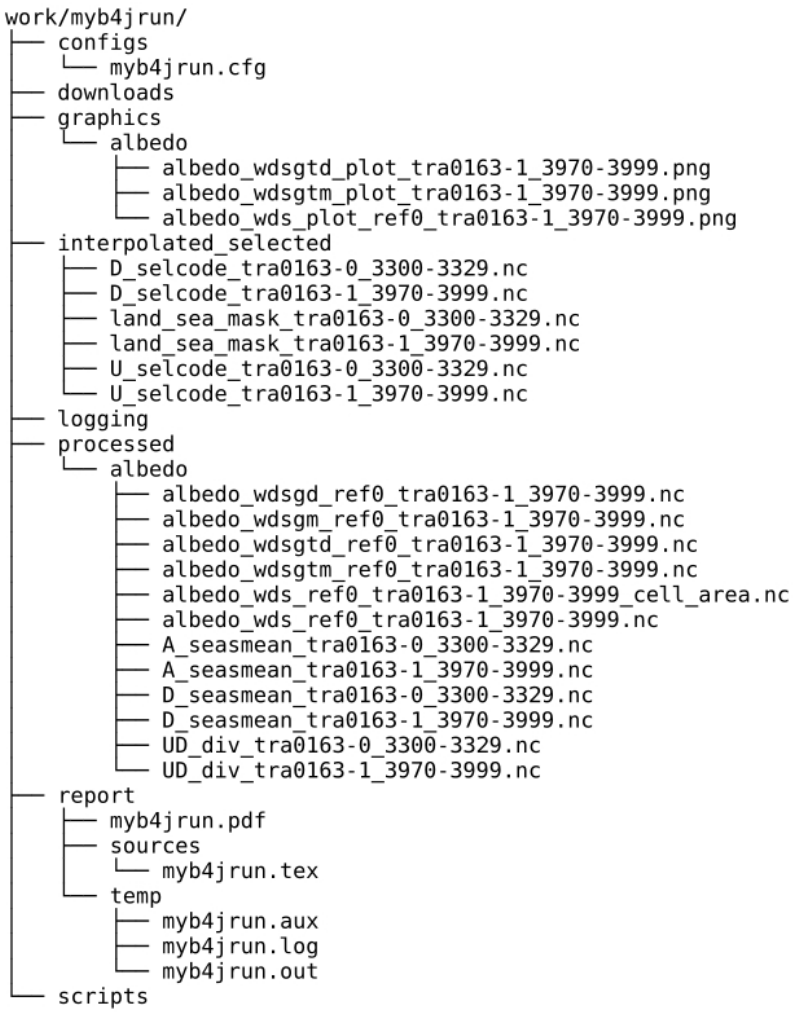


Fig. 4: **Figure:** After the run, structure of `bench4jsbach/work/myb4jrun`

The generated report `myb4jrun.pdf` can be found in `myb4jrun/report`, LaTeX source and temporary files are kept in subdirectories, the graphics included in the report are in a parallel directory `myb4jrun/graphics`. If you want to have a closer look on the diagnostics made, you find the results of the calculations and also the intermediate, preprocessed and downloaded data in the directories `downloads`, `interpolated_selected` respectively `processed`.

References

User guide for developing diagnostic fields

In the following we show how a diagnostic for `experiment` comparison from external sources can be prepared to serve as `diagnostic field` that may be triggered by a **bench4jsbach** run. A `diagnostic field` consists of scripts for the processing chain (preprocessing, analysing, plotting), declarations also for data retrieval and interpolation, descriptive textblocks for generating reports and a configuration file to concentrate the setup. The scripts may be written in any programming language that is supported by your platform. A script has to be layed out for one candidate and one reference so that it is scalable, i.e. it can be called in loops over multiple `experiment` s. It will work on local data which the tool has selected and interpolated from the `experiment` output beforehand as it had been indicated

by the user and the experiment type. The interface of each script has to be declared in a respective section of the diagnostic field s' configuration file. The information to provide include input and output filenames and possibly hints on format, grid, spatial resolution, tiling and global fragmentation and time resolution and period.

We use the example myb4jrun from the last section *User guide for benchmarking* again. In the run we want to include some diagnostic and benchmarking for a clear cut out example scientific issue called myfield.

Declaration in main configuration

The central entry point for a diagnostic field into the run is its *Diagnostic field section* in the main configuration file, in this example myb4jrun.cfg. We declare a subsection called myfield and fill in the path to a new field configuration file as value for the key config_filename, here mypath/myfield.cfg.

```
### diagnostic fields

[fields]
    [[myfield]]
        config_filename=mypath/myfield.cfg
```

Listing: Declaration of myfield in main configuration file myb4jrun.cfg

Configuration file myfield.cfg

In this chapter we explain in detail how to develop the field configuration file. The file has to contain all the settings and communication parameters that are needed for a diagnostic field, from fetching experiment output data until writing the benchmark results in a final report.

Top level configuration

Top level in the diagnostic field configuration file:

key		value	
name	description	datatype	default
field_name	field id conform to id/name of diagnostic field in main configuration	string	-
suffix	output format suffix for processed data of the run	string	-
frequency	output frequence to use in the run	option (yearly, monthly, 6-hourly)	monthly
documentation	self documentation	string	None

For verification it will be checked if the value for the key field_name is consistent with the name given for the field in the main configuration.

In our example we fill in the respective values.

```
field_name=myfield
suffix=nc
frequence=monthly
documentation="""
NAME
        myfield_diagnostic

DESCRIPTION
```

```
        This configuration ...

DETAILS
        For all experiments ...
        """
```

Listing: Example `myfield.cfg`, top level configuration

Processing chain

The processes involved in a `diagnostic field` are interconnected by I/O dependencies. For the time being they have a binding order: 1. `fetch`, 2. `interpolate`, 3. `preprocess`, 4. `diagnose`, 5. `plot`. **bench4jsbach** works internally with an indexing system in which files are addressed by unique infixes, f.e. `VARNAME − INFIX`. For each file that shall be written by a process, the appropriate section has to hold a subsection [[`VARNAME − INFIX`]] which contains the detailed configuration. The infix will also be used as substring for the corresponding filename. If the file shall be read by a subsequent process, it has to be appended to the respective `input` list. For illustration the listing *I/O dependencies* at the bottom of the page shows a detail from the configuration file for the `diagnostic field myfield`. In HTML the graphic serves also as a link to another picture, showing how the I/O dependencies are taken into account.

Fetch section

The beginning of the section is marked by the header [`fetch`]. For each variable that the `diagnostic field` requires from the `experiment` output the **bench4jsbach** run will write a file containing the selected data field. You have to define a subsection f.e. [[`VARNAME − INFIX`]] for it.

A single subsection in section `fetch` contains the following key:value entries:

key		value	
name	description	datatype	default
longname	complete name	string	None
unit	physical unit	string	-

Inside there has to be one subsubsection for each `experiment_type` given for any item in the `experiment` section of the main configuration. This subsubsection must be labelled with the respective `type` name.

Subsubsection for the `experiment type` `MPIESM1`:

key		value	
name	description	datatype	default
name	name of variable to select	string	-
code	code number of variable to select	string	-
submodel	submodel identifier in the original context	string	-
stream	stream identifier in the original context	string	-

For clarification we show one variable subsection labelled `D_selcode` of the `fetch` section from our example `mypath/myfield.cfg`.

```
[fetch]
    [[D_selcode]]
        longname=Surface Downwelling Solar Radiation [W m-2]
        unit=Wm**(-2)
        [[[MPIESM1]]]
            name=swdown_acc
            code=21
            submodel=jsbach
            stream=jsbach
```

Listing: Example `fetch` section configuration

Processing sections

In order to do the preprocessing, diagnostic and plotting **bench4jsbach** calls the executable script given for the key `exec` in the respective section. In the standard case the tool would look in `bench4jsbach/prod/scripts` for it but you may just as well provide another path. We choose again `mypath` as path for the executable scripts in the example below, without limiting the generality.

Preprocessing and diagnostic sections

Top level in the `preprocessing` and the `diagnostic` sections:

key		value	
name	description	datatype	default
exec	executable script to call in **bench4jsbach** run	string	-
input	input files needed by script	stringlist	-

In the example below the script `myfield_prep.py` shall do the `preprocessing`. It has to read a data file that ought to be found during the **bench4jsbach** run by the identifier `D_selcode` as it is given in the value list of the key `input`. It shall store intermediate results in an output file that can be indexed by `D_seasmean` . This is indicated by the appropriatly labelled subsection.

```
[preprocessing]
    exec=mypath/myfield_prep.py
    input=D_selcode,
    [[D_seasmean]]
```

Listing: Example `preprocessing` section configuration

Plotting section

For the plotting we allow several scripts, the section was adjusted to this requirement.

Top level of the `plotting` section:

key		value	
name	description	datatype	default
suffix	output format string	string	-

There are subsections for one or more plotting scripts labelled with a unique identifier for the particular script.

Single plotting script subsection:

key		value	
name	description	datatype	default
exec	executable script to call in **bench4jsbach** run	string	-
input	input files needed by script	stringlist	-
nref	as indicated by plot: comparison with single or all reference experiment	option (single, multiple)	-
plot_title	title string for plot	stringlist	-

As in the other processing sections output files are introduced by labelled subsections beneath. In the example below the subsection `wds_plot` indicates plotting with the script `myfield_wds_plot.py`. According to the value `single` for the key `nref` the script will run seperatly for each candidate, reference `experiment` pair. It shall read a file indexed

by `myfield_wds` which has to be written by a predecessor process. For each pair it is supposed to store a plot output file with the format `png` and a title generated from the output infix `myfield_wds_plot`. For the plots the `plot_title` has to be used, here consisting of a template and parameters, which should be expanded by the script.

```
[plotting]
    suffix=png
    [[wds_plot]]
        exec=myfield_wds_plot.py
        input=myfield_wds,
        nref=single
        plot_title='$\Delta_{C_{\%d}R_{\%d}}$',can_no,ref_no,
        [[[myfield_wds_plot]]]
```

Listing: Example `plotting` section configuration

I/O dependencies

The listing below shows an extract of the example `diagnostic field` configuration file `myfield.cfg`. In HTML the listing is also a link to an illustration of the processing chain formed by I/O dependencies. In the example the file indexed `wds_plot` being output of the plotting process is the end of a chain, started with the file `D_select` which shall contain `experiment` output data selected by the **bench4jsbach** run.

```
[fetch]
    [[D_select]]
        [[[MPIESM1]]]
            code=21
            submodel=jsbach
            stream=jsbach
    [[U_select]]
        [[[MPIESM1]]]
            code=22
            submodel=jsbach
            stream=jsbach

[interpolation]

[preprocessing]
    exec=prep.ex
    input=D_select,U_select,
    [[UD_div]]
    [[A_seasmean]]
    [[D_seasmean]]
                                ...
```

```
...
[diagnostic]
    exec=diagnostic.ex
    input=A_seasmean,D_seasmean,
    [[alb_wds]]
    [[alb_wdsgm]]
    [[alb_wdsgtm]]
        metric=True

[plotting]
    suffix=png
    [[wds_plot]]
       exec=plot_wds.ex
       input=alb_wds,
    [[wdsgtm_plot]]
       exec=plot_wdsgtm.ex
       input=alb_wdsgm,alb_wdsgtm,

[report]
```

Figure: Example `myfield.cfg` (HTML link)

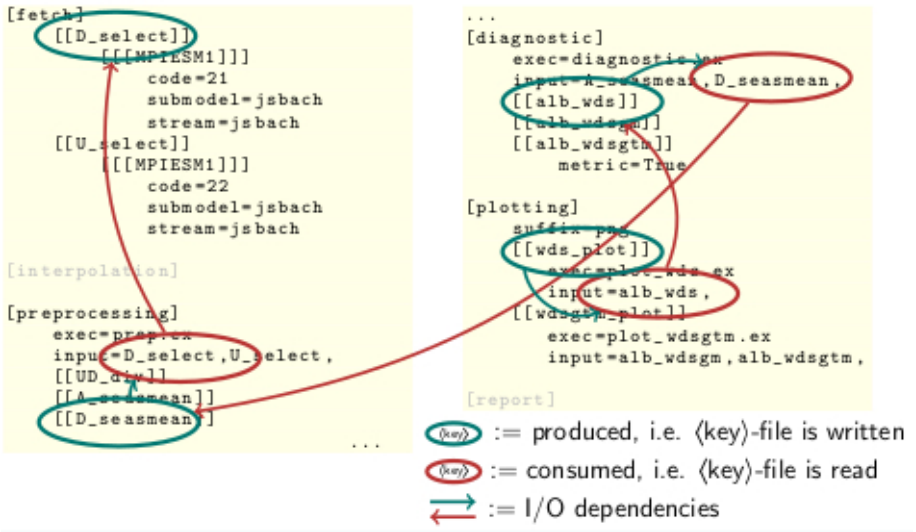


Figure: Highlighted I/O dependency chain for `wds_plot` indexed plot file

Report section

The report section is arranged at the end of the configuration file to improve readabiliy.

Top level in the `report` section:

key		value	
name	description	datatype	default
title	title for the `diagnostic field` ‘s section in the report	string	-
abstract	scientific description of the `diagnostic field`	string	-
technical_TeXed	technical description of the `diagnostic field`	string	-

There have to be subsections for each diagnostic result to appear in the report. For the time being the mapping is based on the conformity with the output file infixes of the `diagnostic` section.

Subsections refering to `diagnostic` output:

key		value	
name	description	datatype	default
title	title of report subsection	string	-
abstract	scientific description of diagnostic method	string	-
technical_TeXed	technical description of diagnostic method	string	-
name	name of diagnostic method	string	-
isbench	switch: Is diagnostic method used as benchmark?	boolean	-
benchvar	name of displayed variable in plotfile	string	-
benchunit	physical unit of displayed variable	string	-
type	type of plot	option (map, metric, barchart)	-

Processing scripts

The processing scripts (for preprocessing, diagnostic and plotting) are run as Python subprocesses. A string list is passed as argument. Each element has the format

```
"XXX:YYYY"
```

with “XXX” being a key out of the main or the `diagnostic field` configuration and “YYY” the corresponding value. The informtion that should be provided include

- attributes of the candidate `experiment`
- attributes of the reference `experiment`
- paths to input files
- paths to output files

The processes have to parse this string list at runtime to get the keyword arguments.

Code documentation

This page provides an auto-generated summary of bench4jsbach’s code documentation.

Bench

The module contains bench related functionality and data.

Functions

bench.get_git_origin()
bench.get_git_branch()
bench.get_git_sha()
bench.get_git_status()
bench.get_ip_address()

bench.get_git_origin

bench.**get_git_origin**()

bench.get_git_branch

bench.**get_git_branch**()

bench.get_git_sha

bench.**get_git_sha**()

bench.get_git_status

bench.**get_git_status**()

bench.get_ip_address

bench.**get_ip_address**()

Creating a bench object

bench.Bench_Object(config_arg, **kwargs)	This object coordinates the bench4jsbach run.

bench.Bench_Object

class bench.**Bench_Object**(*config_arg*, ***kwargs*)

Bases: *diagnostics.Diagnostics*, *report.Report*

This object coordinates the bench4jsbach run.

__init__(*config_arg*, ***kwargs*)

This object coordinates the bench4jsbach run.

Args:

config_arg (str): If config_arg is a local path to an existing bench4jsbach configuration file a run is started. Else config_arg is taken as the name for a new bench4jsbach environment, this is initialized and the configuration file is generated.

Methods

__init__(config_arg, **kwargs)	This object coordinates the bench4jsbach run.
diagnostic_generic(fieldname)	
get_b4j_path(filename, b4j_filetype)	
init_diagnostics()	This method is used to initialize the diagnostic fields.
report_diagnostic()	
write_description()	
write_report()	
write_summary()	

Attributes

TeXlib_dirtree
TeXlib_forest
bench_configs_path
bench_name
bench_path
config
configs_path
diagnostic_fields
download_path
experiments
graphics_path
interpolated_selected_path
logfile_path
logger
logger_name
logging_path

Continued on next page

Table 4 – continued from previous page

n_cans
n_refs
path_template
processed_path
report_path
reporting
scripts_path
wikipage

Diagnostics

diagnostics	The module contains diagnostic related classes, functionality and data.

diagnostics

The module contains diagnostic related classes, functionality and data.

Functions

find_files(pattern, path[, single, raise_e])
get_largest_common_yearlist(filelist)
get_seasonal_period_2d_coords(xvals, yvals, ...)
get_vardes_from_codes(exp_name, code, path)
print_item_or_list_rec(something, indent)
subprocessing(cmd[, exit_on_error, ...])
teXify(string)
touch(filename)

Classes

CMIP5_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations done for CMIP5.
Cdo([returnCdf, returnNoneOnError, ...])	
Diagnostics(**kwargs)	
MPIESM1_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations with MPIESM1.
OrderedDict(*args, **kwds)	Dictionary that remembers insertion order
Set([iterable])	Mutable set class.
datetime(year, month, day[, hour[, minute[, ...)	The year, month and day arguments are required.

Creating a diagnostics

diagnostics.Diagnostics(**kwargs)

diagnostics.Diagnostics

class diagnostics.**Diagnostics**(***kwargs*)

Bases: object

__init__(***kwargs*)

Methods

__init__(**kwargs)
diagnostic_generic(fieldname)

Attributes

path_template

Experiment

experiment	Module contains experiment related classes, methods and metadata.

experiment

Module contains experiment related classes, methods and metadata.

Functions

find_files(pattern, path[, single, raise_e])	Function to find files with a name containing a given infix.
getdomain()	
gethostname()	
print_none(item)	
subprocessing(cmd[, exit_on_error, ...])	

Classes

CMIP5_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations done for CMIP5.
Cdo([returnCdf, returnNoneOnError, ...])	
Dataset	A netCDF *netCDF4.Dataset* is a collection of dimensions, groups, variables and att
Experiment(b4j_obj, **kwargs)	Base class representing methods and metadata of an experiment.
MPIESM1_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations with MPIESM1.

Functions

experiment.find_files(pattern, path[, ...])	Function to find files with a name containing a given infix.

experiment.find_files

experiment.**find_files**(*pattern*, *path*, *single=True*, *raise_e=False*)

Function to find files with a name containing a given infix.

Creating an experiment

experiment.Experiment(b4j_obj, **kwargs)	Base class representing methods and metadata of an experiment.
experiment.CMIP5_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations done for CMIP5.
experiment.MPIESM1_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations with MPIESM1.

experiment.Experiment

class experiment.**Experiment**(*b4j_obj*, ***kwargs*)

Bases: object

Base class representing methods and metadata of an experiment.

Specialized classes for model simulations and observational datasets are derived from it.

__init__(*b4j_obj*, ***kwargs*)

Objects are initialized giving their attributes.

In a bench4jsbach run this is done by the runskript reading a configuration file.

Methods

__init__(b4j_obj, **kwargs)	Objects are initialized giving their attributes.
accumulate_streams(orig_dirpath, ...)	
fetch_archived_streams(fetchfile_list)	
fetch_land_sea_mask(origin_path, ...)	
fetch_outdata()	Fetch outdata.
remotehost_fetching(remote_path, target_path)	
remotehost_processing(cmd)	
teX_repr()	
teX_short_repr([shortname])	

Attributes

B4J_atmres
B4J_canno
B4J_endyear
B4J_etype
B4J_finalyear
B4J_host
B4J_id
B4J_index
B4J_initialyear
B4J_iscan

Continued on next page

Table 17 – continued from previous page

B4J_isref	
B4J_landseamask	
B4J_name	
B4J_name_orig	
B4J_obj	
B4J_oceres	
B4J_path	this is a comment on B4J_path
B4J_refno	
B4J_startyear	

experiment.CMIP5_Experiment

class experiment.**CMIP5_Experiment**(*b4j_obj*, ***kwargs*)

Bases: *experiment.Experiment*

Derived class specialized on simulations done for CMIP5.

__init__(*b4j_obj*, ***kwargs*)

Methods

__init__(b4j_obj, **kwargs)	
accumulate_streams(orig_dirpath, ...)	
fetch_archived_streams(fetchfile_list)	
fetch_land_sea_mask(origin_path, ...)	
fetch_outdata()	Fetch outdata.
remotehost_fetching(remote_path, target_path)	
remotehost_processing(cmd)	
teX_repr()	
teX_short_repr([shortname])	

Attributes

B4J_atmres
B4J_canno
B4J_endyear
B4J_etype
B4J_finalyear
B4J_host
B4J_id
B4J_index
B4J_initialyear
B4J_iscan
B4J_isref
B4J_landseamask
B4J_name
B4J_name_orig
B4J_obj

Continued on next page

Table 19 – continued from previous page

B4J_oceres
B4J_path
B4J_refno
B4J_startyear
suffix

experiment.MPIESM1_Experiment

class experiment.**MPIESM1_Experiment**(*b4j_obj*, ***kwargs*)

Bases: *experiment.Experiment*

Derived class specialized on simulations with MPIESM1.

__init__(*b4j_obj*, ***kwargs*)

Methods

__init__(b4j_obj, **kwargs)	
accumulate_streams(orig_dirpath, ...)	
fetch_archived_streams(fetchfile_list)	
fetch_land_sea_mask(origin_path, ...)	
fetch_outdata()	Fetch outdata.
remotehost_fetching(remote_path, target_path)	
remotehost_processing(cmd)	
teX_repr()	
teX_short_repr([shortname])	

Attributes

B4J_atmres
B4J_canno
B4J_endyear
B4J_etype
B4J_finalyear
B4J_host
B4J_id
B4J_index
B4J_initialyear
B4J_iscan
B4J_isref
B4J_landseamask
B4J_name
B4J_name_orig
B4J_obj
B4J_oceres
B4J_path
B4J_refno
B4J_startyear

Continued on next page

Table 21 – continued from previous page

codesfiles_dict
outfiles_dict
pool_dict

Report

report	The module contains report related classes, functionality and data.

report

The module contains report related classes, functionality and data.

Functions

get_filesystem_tree_dirtree(rootDir[, onlydirs])
get_filesystem_tree_tikz(rootDir[, onlydirs])
includegraphics_template(path[, width, hspace])
lib_avail(libname)
print_item_or_list_rec(something, indent)
print_none(item)
subprocessing(cmd[, exit_on_error, ...])
teXify(string)

Classes

Dataset	A netCDF *netCDF4.Dataset* is a collection of dimensions, groups, variables and attributes.
OrderedDict(*args, **kwds)	Dictionary that remembers insertion order
Report()	

Creating a report

report.Report()

report.Report

class report.**Report**

Bases: object

__init__()

Methods

__init__()

Continued on next page

Table 26 – continued from previous page

report_diagnostic()
write_description()
write_report()
write_summary()

Attributes

TeXlib_dirtree
TeXlib_forest
diagnostic_fields

Wikipage

wikipage	The module contains redmine related classes, functionality and data.

wikipage

The module contains redmine related classes, functionality and data.

Classes

Bench4JSBach_Wikipage(redmine_user, redmine_key)	
Redmine(url, **kwargs)	An entry point for all requests

Creating a wikipage

wikipage.Bench4JSBach_Wikipage(redmine_user, ...)

wikipage.Bench4JSBach_Wikipage

class wikipage.**Bench4JSBach_Wikipage**(*redmine_user*, *redmine_key*)

Bases: object

__init__(*redmine_user*, *redmine_key*)

Methods

__init__(redmine_user, redmine_key)
fetch_simulation_row(experiment_role, ...)
fetch_simulation_row_split_period(...)
test()
upload_file(filepath)
write_b4j_row()

Attributes

codezmaw_url
project_name
redmine
redmine_project
redmine_wikipage
wikipage_name

Util

util	Module contains functions and constants of general nature.

util

Module contains functions and constants of general nature.

Functions

get_filesystem_tree_dirtree(rootDir[, onlydirs])
get_filesystem_tree_tikz(rootDir[, onlydirs])
getdomain()
gethostname()
pprint_class_member_methods(modulename, ...)
print_item_or_list_rec(something, indent)
print_none(item)
setup_logger(*args, **kwargs)
str2bool(s)
subprocessing(cmd[, exit_on_error, ...])
touch(filename)

Classes

WrappedFixedIndentingLog([fmt, datefmt, ...])

References

[GiESM1] Giorgetta, M., et al.: Climate and carbon cycle changes from 1850 to 2100 in MPI-ESM simulations for the coupled model intercomparison project phase 5. Journal of Advances in Modeling Earth Systems, 5, 572-597 , 2013, doi:10.1002/jame.20038

[ReJSBA] Reick, C., et al.: Representation of natural and anthropogenic land cover change in MPI-ESM. Journal of Advances in Modeling Earth Systems, 5, 459-482 , 2013, doi:10.1002/jame.20022

Python Module Index

d

diagnostics, 19

e

experiment, 20

r

report, 24

u

util, 26

w

wikipage, 25

Index

Symbols

__init__() (bench.Bench_Object method), 18
__init__() (diagnostics.Diagnostics method), 20
__init__() (experiment.CMIP5_Experiment method), 22
__init__() (experiment.Experiment method), 21
__init__() (experiment.MPIESM1_Experiment method), 23
__init__() (report.Report method), 24
__init__() (wikipage.Bench4JSBach_Wikipage method), 25

B

Bench4JSBach_Wikipage (class in wikipage), 25
Bench_Object (class in bench), 18

C

CMIP5_Experiment (class in experiment), 22

D

Diagnostics (class in diagnostics), 20
diagnostics (module), 19

E

Experiment (class in experiment), 21
experiment (module), 20

F

find_files() (in module experiment), 21

G

get_git_branch() (in module bench), 17
get_git_origin() (in module bench), 17
get_git_sha() (in module bench), 17
get_git_status() (in module bench), 17
get_ip_address() (in module bench), 17

M

MPIESM1_Experiment (class in experiment), 23

R

Report (class in report), 24
report (module), 24

U

util (module), 26

W

wikipage (module), 25